



華南師範大學

本科学生实验（实践）报告

院 系：计算机学院

实验课程：编译原理实验

实验项目：实验 4：LR(1)分析生成器

指导老师：黄煜廉

开课时间：2025 ~ 2026 年度第 1 学期

专 业：计算机科学与技术

班 级：计科 2 班

学生姓名：袁知本

华南师范大学教务处

华南师范大学实验报告

学生姓名 袁知本 学 号 20232131006
专 业 计算机科学与技术 年级、班级 2023 级 计科 2 班
课程名称 编译原理实验 实验项目 实验 4: LR(1)分析生成器
实验类型 ☐验证 ☐设计 ☐综合 实验时间 2025 年 5 月 28 日
实验指导老师 黄煜廉 实验评分 _____

一、实验内容

设计一个应用软件，以实现 LR(1)分析生成器

必做内容：

1. 要提供一个文法输编辑界面，让用户输入文法规则（可保存、打开存有文法规则的文件）；
2. 求出文法各非终结符号的 first 集合与 follow 集合，并提供窗口以便用户可以查看这些集合结果。【可以采用表格的形式呈现】；
3. 需要提供窗口以便用户可以查看文法对应的 LR(0)DFA 图。（可以用画图的方式呈现，也可用表格方式呈现该图点与边数据）；
4. 判断该文法是否为 SLR(1)文法。（应提供窗口呈现判断的结果，若不是 SLR(1)文法，需要在窗口中显示原因）；
5. 需要提供窗口以便用户可以查看文法对应的 LR(1)DFA 图。（可以用画图的方式呈现，也可用表格方式呈现该图点与边数据）；
6. 需要提供窗口以便用户可以查看文法对应的 LR(1)分析表。【LR(1)分析表采用表格的形式呈现】；
7. 应用程序应为 Windows 界面（窗口式图形界面）

选做内容:

1. 需要提供窗口以便用户驶入需要分析的句子;
2. 需要提供窗口以便用户查看使用 LR(1)分析该句子的过程。【可以使用表格的形式逐行显示分析过程】;
3. 书写完善的软件文档

二、实验目的

1. 实现一个带图形化界面的 LR(1)分析生成器;
2. 理解上下文无关文法 (2 型文法) 在语法分析中的作用;
3. 掌握 FIRST 集合、FOLLOW 集合的计算方式和计算过程;
4. 掌握 LR(0)、SLR(1)、LR(1)自动机 (DFA 图) 的构造方式机器分析表的构建方法, 并能够区分出 LR(0)、SLR(1)、LR(1)三者之间的差异;
5. 掌握文法分析表的构造与判断分析表的合法性的基本原则

三、实验文档:

注: 本次实验中的必做内容均已完成, 同时也完成了全部的选做内容, 选做内容请查看选作内容 1 和选做内容 2 文件夹

1. 实验需求分析

本实验的目标是用 C++语言实现一个具有图形化界面的 LR(1)分析生成器软件, 要为用户实现文件对话框选择和直接输入等多种输入文法规则的方式, 需实现由用户选择文法规则文件保存功能, 还要实现 FIRST 集合、FOLLOW 集合、LR(1)分析表的表格化展示, 以及 LR(0)、LR(1)的 DFA 图绘制、SLR(1)文法判断等功能, 同时, 通过 QT 平台构建一个用户友好型可视化图形界面

在选做内容上, 还要完成用户输入当前文法对应的语句, 并将该语句使用 LR(1)进行分析的过程表格化展示

功能需求

1) 用户界面设计

使用说明区、文本输入区、FIRST&FOLLOW 集合表格化展示区、DFA 图展示区、SLR(1)文法判断区、LR(1)分析展示

区共 6 个模块：

- **使用说明区：**对该软件的使用方式进行说明的区域，告知用户语法规则、句子输入的要求和格式，以及展示内容中的一些解释
- **文本输入区：**包括语法输入区和句子输入区，语法规则的输入可以通过键盘输入，也可以通过打开文件对话框，从本地选取语法规则文件载入，并且可以将档期那输入到输入区的语法规则保存到本地；句子输入区目前可通过直接键盘输入
- **FIRST&FOLLOW 集合表格化展示区：**借助 Qt 中的 TableView 控件，实现处理语法规则后用表格的形式展示语法中每个非终结符的 First 集合和 Follow 集合，该功能由用户点击按钮【查看 first 集合和 follow 集合】实现
- **DFA 图展示区：**该区域包含 LR(0)DFA 图展示区和 LR(1)DFA 图展示区，均借助 Qt 中的 GraphicsView 控件，实现当前语法的 LR(0)和 LR(1)DFA 图的绘制与展示，该功能由用户点击按钮【查看 LR(0)DFA】和【查看 LR(1)DFA】实现
- **SLR(1)文法判断区：**该区域将会对当前用户提供的语法规则进行“移进-规约”和“规约-规约”冲突检测，以判断当前语法是否为 SLR(1)文法，并将判断结果输出显示，该功能通过用户点击按钮【SLR(1)判断】实现
- **LR(1)分析展示区：**该区域包含 LR(1)分析表展示区和 LR(1)句子分析过程表展示区两部分，前者借助 Qt 中的 TableView 控件，可让用户查看当前语法的 LR(1)分析表；后者同样借助 Qt 中的 TableView 控件，让用户可以查看到使用 LR(1)分析用户提供的句子的过程，这两项功能分别通过按钮【查看 LR(1)分析表】和【LR(1)句子

分析】触发

2) 语法规则处理

本次实验中的语法规则需要从 Qt 控件 PlainTextEdit 中获取，获取后按照“按行拆分->解析每条语法规则（区分 LHS 和 RHS，处理分支）->增广处理->编号”的过程进行语法规则处理

3) 求 First 集合和 Follow 集合

通过刚才的语法处理，我将处理后的语法按照左部和右部区分，存放到一个 unordered_map 容器变量 grammar_map 中，然后从该容器中遍历所有的语法，遍历右部所有分支，并获取每个分支的分词，判断是否为终结符（串），若是，则加入当前遍历的左部非终结符（串）的 first 集合中；若不是，则从这个非终结符继续遍历其右部产生式，将这个非终结符的 first 集合去除 ϵ 空串后添加到当前左部非终结符的 first 集合中，若所有符号都能推导到 ϵ ，则将 ϵ 添加到当前非终结符（串）的 first 集合中，就这样循环遍历直至所有语法遍历完为止；

对于 follow 集合，我将处理后的语法按照用户输入的顺序放入到一个 queue 容器变量 grammar_deque 中，此时遍历该容器，将每个非终结符（串）对应的产生式序列获取出来，遍历其中每个非终结符（串）B，计算其 follow 集合

4) 构建 DFA 图

构建 LR(0)的 DFA 图，需要借助闭包计算函数 closure 和 GOTO 运算函数，前者确定 LR(0)每个项目集中的项目，将所有可通过闭包运算扩展的项目都加入到项目集中；后者计算 LR(0)项目在某个符号 X 上的 GOTO，获取通过某个符号到达的新项目集，得到所有项目集后，将其转换为 DFA 图；

构建 LR(1)的 DFA 图，需要借助闭包计算函数 closure_LR1

和 GOTO_LR1 运算函数，总体的过程和 LR(0)的类似，但是 LR(1)要考虑到展望字符（串）

5) SLR(1)文法分析判断

SLR(1)文法分析判断，需要进行冲突检测，这将通过函数 SLR1_check1()和 SLR1_check2()进行，检测过后将检测结果输出显示出来供用户查看

6) LR(1)分析表的构建

借助闭包计算函数 closure 和 GOTO 运算函数获取 LR(1)所有的项目集，之后通过表格的形式显示出来即可

7) LR(1)分析句子

用户通过键盘输入要分析的句子，程序通过处理用户给的文法规则，构建 LR(1)项目集，然后对用户输入的句子进行 LR(1)分析，将分析处理的过程通过表格的形式展现出来

2. 实验环境

硬件设备：基于 x64 的处理器，RAM16.0GB

软件设备：Windows11 家庭版，

Qt5.14.2，

Qt Creator 4.11.1(Community)

3. 系统设计

1) 简洁方便的交互界面：

用 Qt 开发的可视化交互界面如下图所示：

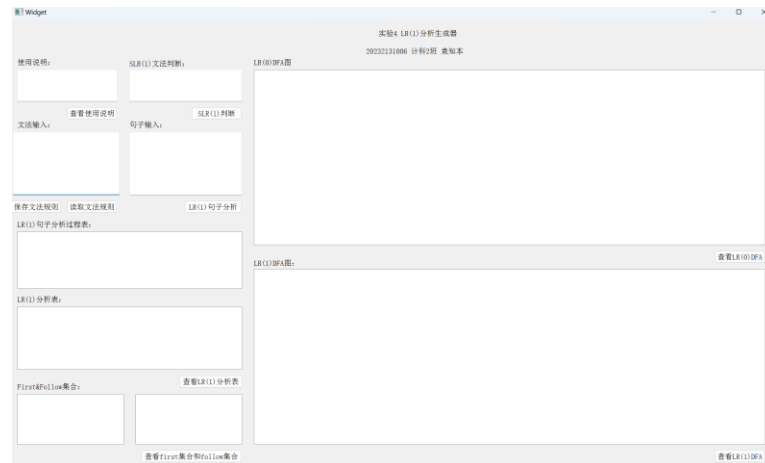


图 3-1-1 交互界面

2) 文法规则的处理:

处理文法规则，需要如下的数据结构定义：

```

35 // 文法规则字符串
36 string grammar_str;
37
38 // 结构化文法规则map（非终结符->所有右部分支）
39 unordered_map<string, set<string>> grammar_map;
40
41 // 文法规则集合（顺序id、左部、右部）
42 struct grammarUnit
43 {
44     int gid;
45     string left;
46     string right;
47     grammarUnit(string l, string r): gid(-1), left(l), right(r){}
48 };
49
50 // 文法数组（按照原先输入的文法规则顺序存储增广和划分后的文法规则）
51 deque<grammarUnit> grammar_deque;
52
53 // 起始符、增广后的起始符
54 string startSymbol, trueStartSymbol;

```

图 3-2-1 文法规则数据结构定义

从 Qt 控件 PlainTextEdit 中获取用户提供的文法规则，获取后按照“按行拆分->解析每条文法规则（区分 LHS 和 RHS，处理分支）->增广处理->编号”的过程进行文法规则处理，grammar_str 字符串用于存储用户提供的全部文法规则，grammar_map 用于存储区分处理后的文法的 LHS 和对应的 RHS，grammar_deque 用于按用户给出的文法规则的顺序保存文法

处理文法规则，还需要用到 isNonTerminalName、splitAlternatives 这两个函数，前者用于判断知否为终结符（串）；后者用于即将文法规则按照或运算符符号 ‘|’ 划

分出多个分支：

```
149 // 判断是否为非终结符（串）（允许大小写字母、数字和下划线，首字符为字母）
150 bool isNonTerminalName(const string& str)
151 {
152     // 字符串为空或首字符不是字母的情况
153     if (str.empty() || !isalpha(str[0])) return false;
154     // 遍历串内所有字符，考虑字符不是字母、数字或下划线的情况
155     for (char c : str)
156         if(!isalnum(c) || c == '_') return false;
157     return true;
158 }
159
160 // 将二型文法RHS按'|'划分为多个分支，并返回所有分支（仅右部产生式）
161 vector<string> splitAlternatives(const string& rhs)
162 {
163     vector<string> parts;
164     string part;
165     istringstream ss(rhs);
166     // 先通过字符串流ss将rhs读入到parts中，再判断是否有'|'符号
167     while(getline(ss, part, '|'))
168     {
169         // 去掉前后空格
170         auto l = part.find_first_not_of(" \t");
171         auto r = part.find_last_not_of(" \t");
172         // 将每次划分得到的非空左部分添加到vector中暂存
173         if (l != string::npos)
174             parts.push_back(part.substr(l, r - l + 1));
175     }
176     return parts;
177 }
```

图 3-2-2 isNonTerminalName 函数和 splitAlternatives 函数

以下为文法处理函数：

```
240 // 文法规则处理函数
241 void handleGrammar()
242 {
243     // first step: 按行拆分
244     // 将整个多行的文法规则字符串grammar_str按行拆分，去掉空行后存入lines
245     vector<string> lines;
246     istringstream iss(grammar_str);
247     string line;
248     while(getline(iss, line))
249     {
250         if(!line.empty())
251             lines.push_back(line);
252     }
253 }
```

图 3-2-3 第一步按行拆分


```

254 // second step: 解析每条文法规则
255 bool firstRule = true;
256 for (auto& line : lines)
257 {
258     // 找到符号"->", 通过符号"->"将每行文法规则划分为左部和右部 (LHS 和 RHS)
259     auto pos = line.find("->");
260     // 跳过不合法行 (找不到符号"->")
261     if(pos == string::npos) continue;
262
263     // LHS 和 RHS, 划分左右段
264     string lhs = line.substr(0, pos);
265     string rhs = line.substr(pos + 2);
266
267     // 去除左右空格
268     auto trim = [](string& s)
269     {
270         auto l = s.find_first_not_of(" \t");
271         auto r = s.find_last_not_of(" \t");
272         if (l == string::npos)
273         {
274             s.clear();
275             return;
276         }
277         s = s.substr(l, r - l + 1);
278     };
279     trim(lhs);
280     trim(rhs);
281
282     // 判断是否为合法非终结符 (串)
283     if (!isNonTerminalName(lhs))
284     {
285         // 后续有待增加其他可视化提示
286         cerr << "Error: 非法的非终结符名字" << lhs << endl;
287         continue;
288     }
289
290     // 记录开始符号
291     if(firstRule)
292     {
293         startSymbol = lhs;
294         trueStartSymbol = lhs;
295         firstRule = false;
296     }
297
298     // 拆分多分支 (将或运算中的每条分支单独拆分出来), 若无分支则直接存储
299     auto alts = splitAlternatives(rhs);
300     for(string& alt : alts)
301     {
302         grammar_map[lhs].insert(alt);
303         grammar_deque.emplace_back(lhs, alt);
304     }
305 }

```

图 3-2-4 第二步解析每条文法规则

```

307 // third step: 增广, 若开始符号有多条产生式, 则加"^"开头的新开始, 确保总的开始符号有且仅有一个
308 // 增广后统一用"^"作为总的开始符号
309 // grammar_map[startSymbol].size() > 1 意味着开始符号 (串) 不唯一
310 if(grammar_map[startSymbol].size() >= 1)
311 {
312     grammar_deque.push_front(grammarUnit("^", startSymbol));
313     trueStartSymbol = "^";
314 }

```

图 3-2-5 第三步增广

```

316 // forth step: 编号
317 int gid = 0;
318 for (auto& unit : grammar_deque)
319     unit.gid = gid++;
320 }

```

图 3-2-6 第四步编号

3) 获取 FIRST 集合和 FOLLOW 集合:

对于 FIRST 集合和 FOLLOW 集合的求解，需要用到如下的数据结构：

```
56 // first 集合
57 unordered_map<string, set<string>> firstSet;
58
59 // follow 集合
60 unordered_map<string, set<string>> followSet;
61 unordered_set<string> nonTerminals;
```

图 3-3-1 存储 first 集合和 follow 集合的数据结构

其中，nonTerminals 是非终结符的集合

对于求一个非终结符的 first 集合，需要考虑下面三种情况：

若右部直接是终结符，则将非终结符加入到 first 集合中；

若右部是非终结符，则将该非终结符的 first 集合去掉空串 ϵ 后加入到 first 集合中；

若所有符号都能推到 ϵ ，则将 ϵ 加入到 first 集合中

以下即为求 first 集合的函数：

```

343 // 求 first 集合
344 void getFirstSet()
345 {
346     // 初始化非终结符集合，将grammar_map中的所有左部非终结符加入到nonTerminals中
347     for(auto& [lhs, rhsSet] : grammar_map)
348         nonTerminals.insert(lhs);
349
350     bool changed;
351     do
352     {
353         changed = false;
354         // 遍历grammar_map中所有的右部产生式
355         for(const auto& [lhs, rhsSet] : grammar_map)
356         {
357             // 遍历右部每个分支
358             for(const string& rhs : rhsSet)
359             {
360                 // 获取当前分支的每个分词
361                 auto symbols = splitSymbols(rhs);
362                 // 标记是否所有符号能推出ε
363                 bool epsilon_in_all = true;
364
365                 // 遍历每个分词（符号（串））
366                 for(const string& symbol : symbols)
367                 {
368                     // 若为终结符（串），则直接加入当前非终结符（串）的first集合中
369                     if(isTerminal(symbol))
370                     {
371                         if(firstSet[lhs].insert(symbol).second)
372                             changed = true;
373                         // 终结符（串）无法推出ε
374                         epsilon_in_all = false;
375                         // 停止对当前产生式的扫描遍历
376                         break;
377                     }
378                     // 若为非终结符（串），则将这个非终结符（串）的 first 集合去除ε后添加到当前非终结符（串）中
379                     else
380                     {
381                         for(const string& fsymbol : firstSet[symbol])
382                         {
383                             if(fsymbol != "ε" && firstSet[lhs].insert(fsymbol).second)
384                                 changed = true;
385                             // 若该非终结符（串）的first集合不含ε，则无需继续遍历后续符号
386                             if(!firstSet[symbol].count("ε"))
387                             {
388                                 epsilon_in_all = false;
389                                 break;
390                             }
391                         }
392                     }
393                 }
394                 // 若所有符号都能推到ε，则将ε加入到当前非终结符（串）的first集合中
395                 if(epsilon_in_all)
396                     if(firstSet[lhs].insert("ε").second)
397                         changed = true;
398             }
399         }
400     } while(changed); // 循环直至无新元素的加入

```

图 3-3-2 求 first 集合的函数

而对于求一个非终结符的 follow 集合（若为终结符，则无需进行 follow 集合求解），有形如 $A \rightarrow cBd$ 的语法规则（此处为求 B 的 follow 集合），需要考虑下面三种情况：

若 d 为终结符，则将 d 加入到 B 的 follow 集合当中；

若 d 为非终结符，则将 $\text{first}(d) - \{\epsilon\}$ 加入到 B 的 follow 集合中；

若 d 可推得 ϵ 或 d 本身为空，则将 $\text{follow}(A)$ 添加到 B 的 follow 集合中

据此，可得如下的 follow 集合求解函数：

```

402 // 求follow集合
403 void getfollowSet()
404 {
405     // 初始化, 将增广文法起始符号的follow集合包含结束符号$
406     followSet[trueStartSymbol].insert("$");
407     bool changed;
408     do
409     {
410         changed = false;
411         // 遍历文法规则序列, 按用户输入的文法规则顺序处理产生式
412         for(const auto& unit : grammar_deque)
413         {
414             const string& lhs = unit.left;
415             // 获取当前非终结符(串)的产生式序列
416             auto symbols = splitSymbols(unit.right);
417
418             // 针对序列中每个非终结符(串)B, 计算其follow集合
419             for(size_t i = 0; i < symbols.size(); ++i)
420             {
421                 string B = symbols[i];
422                 // 若B为终结符(串), 则跳过它
423                 if(!nonTerminals.count(B)) continue;
424
425                 // 用于判断右侧剩余符号是否均可推ε
426                 bool epsilon_suffix = true;
427
428                 // 考察B之后的符号(串) beta
429                 for(size_t j = i + 1; j < symbols.size(); ++j)
430                 {
431                     const string& beta = symbols[j];
432                     epsilon_suffix = false;
433
434                     // 若beta是终结符, 则加入到B的follow集合中
435                     if(isTerminal(beta))
436                     {
437                         if(followSet[B].insert(beta).second)
438                             changed = true;
439                         break;
440                     }
441
442                     // beta为非终结符, 则将first(beta)-{ε}加入到B的follow集合中
443                     for(const string& fsymbol : firstSet[beta])
444                         if(fsymbol != "ε" && followSet[B].insert(fsymbol).second)
445                             changed = true;
446
447                     // beta不含有ε则停止查找
448                     if(!firstSet[beta].count("ε")) break;
449                     // 含有ε则继续查找下一符号(串)
450                     else epsilon_suffix = true;
451                 }
452
453                 // 若beta全可推得ε或beta为空, 则将左部的follow集合加入到B的follow集合中
454                 if(epsilon_suffix)
455                 {
456                     for(const string& f : followSet[lhs])
457                     {
458                         if(followSet[B].insert(f).second)
459                             changed = true;
460                     }
461                 }
462             }
463         }
464     } while(changed); // 循环直至无新元素增加
465 }

```

图 3-3-3 求 follow 集合的函数

在完成对当前文法的 first 集合和 follow 集合的求解后, 通过 Qt 的 TableView 控件, 将这两个集合以表格的形式输出显示出来供用户查看

4) DFA 图绘制:

进行 LR(0)、LR(1)的 DFA 图绘制, 首先需要构建 LR(0)、LR(1)项目集, 需要用到如下的数据结构:

```

64 struct nextStateUnit
65 {
66     // 进入该状态所通过的字符(串)
67     string s;
68     // nextState的id
69     int sid;
70 };
71
72 // LR(0)项目, 形如[A->a.b]
73 struct Item {
74     // gid, 对应 grammar_deque 中产生式的序号
75     int gid;
76     // 右部各符号(串)
77     vector<string> rhs;
78     // "."符号, 用于标志该符号在rhs中的位置
79     int dot;
80     bool operator <(Item const& o) const {
81         if(gid != o.gid) return gid < o.gid;
82         if(dot != o.dot) return dot < o.dot;
83         return rhs < o.rhs;
84     }
85
86     bool operator==(Item const& o) const {
87         return gid == o.gid
88             && dot == o.dot
89             && rhs == o.rhs;
90     }
91 };
92
93 // LR(1)项目, 形如 [A->a.b, c]
94 struct Item_LR1 {
95     // gid, 对应 grammar_deque 中产生式的序号
96     int gid;
97     // 右部各符号(串)
98     vector<string> rhs;
99     // "."符号, 用于标志该符号在rhs中的位置
100     int dot;
101     // 展望符号(串)集合
102     set<string> lookahead;
103
104     bool operator<(Item_LR1 const& o) const {
105         if(gid != o.gid) return gid < o.gid;
106         if(dot != o.dot) return dot < o.dot;
107         if(rhs != o.rhs) return rhs < o.rhs;
108         return lookahead < o.lookahead;
109     }
110     bool operator==(Item_LR1 const& o) const {
111         return gid==o.gid && dot==o.dot && rhs==o.rhs && lookahead==o.lookahead;
112     }
113 };
114
115 // LR(0)自动机状态: 一个项目集合, 使用set便于快速查重
116 using State = set<Item>;
117
118 // LR(1)自动机状态: 一个项目集合, 使用set便于快速查重
119 using State_LR1 = set<Item_LR1>;
120
121 // 状态间的转移: 从状态from经由符号(串)symbol转移到状态to
122 struct Trans {
123     int from;
124     int to;
125     string symbol;
126 };
127
128 // 所有状态(LR0)
129 vector<State> states;
130
131 // 所有状态(LR1)
132 vector<State_LR1> states_LR1;
133
134 // 所有转移(LR0和LR1通用)
135 vector<Trans> trans;

```

图 3-4-1 构建 DFA 图所需的数据结构

构建 LR(0)的 DFA 图, 需要借助闭包计算函数 `closure` 和 `GOTO` 运算函数, 前者确定 LR(0)每个项目集中的项目, 将所有可通过闭包运算扩展的项目都加入到项目集中; 后者计算 LR(0)项目在某个符号 `X` 上的 `GOTO`, 获取通过某个符号到达的新项目集, 得到所有项目集后, 将其转换为 DFA 图, 具体代码如下:

```

654 // LR0生成DFA程序入口
655 int getLR0()
656 {
657     // 初态: closure([S'→. S]), 增广后的开始位置
658     vector<string> startSymbols = splitSymbols(grammar_deque.front().right);
659     Item startItem(grammar_deque.front().gid, startSymbols, 0);
660     states.push_back(closure({startItem}));
661
662     bool added;
663     do
664     {
665         added = false;
666         for(int i = 0; i < (int)states.size(); ++i)
667         {
668             // 对每个可能的符号(串) x (终结或非终结)
669             set<string> symbols;
670             for(auto& it : states[i])
671             {
672                 if(it.dot < (int)it.rhs.size())
673                     symbols.insert(it.rhs[it.dot]);
674             }
675             for(auto& X : symbols)
676             {
677                 State J = GOTO(states[i], X);
678                 if(J.empty()) continue;
679                 // 看 J 是否已存在
680                 auto jt = find(states.begin(), states.end(), J);
681                 int j;
682                 if(jt == states.end())
683                 {
684                     states.push_back(J);
685                     j = states.size() - 1;
686                     added = true;
687                 }
688                 else
689                 {
690                     j = distance(states.begin(), jt);
691                     trans.push_back({i, j, X});
692                 }
693             }
694         }
695     } while(added);
696
697     // 二次去重
698     int max_from = trans[1].from;
699     for (const auto& t : trans)
700         if (t.from > max_from)
701             max_from = t.from;
702
703     trans.erase(
704         std::remove_if(
705             trans.begin(),
706             trans.end(),
707             [max_from](const Trans& t) { return t.from == max_from; }
708         ),
709         trans.end()
710     );
711     return states.size();
712 }

```

图 3-4-2 LR(0)项目集转换图构造

构建 LR(1)的 DFA 图，需要借助闭包计算函数 `closure_LR1` 和 `GOTO_LR1` 运算函数，总体的过程和 LR(0)的类似，但是 LR(1)要考虑到展望符号（串），其具体代码如下：

```

745 // 构建LR1分析表
746 void buildLR1Table() {
747     int N = states_LR1.size();
748     ACTION.assign(N, {});
749     GOTO_.assign(N, {});
750
751     // 1) 先把 trans1 中的所有 (from, symbol)->to 映射缓存起来
752     std::map<std::pair<int, std::string>, int> transMap;
753     for (auto& t : trans) {
754         transMap[{t.from, t.symbol}] = t.to;
755     }
756
757     // 2) 填 ACTION 和 GOTO 表
758     for (int i = 0; i < N; ++i) {
759         // 2.1) ACTION 部分
760         for (auto& it : states_LR1[i]) {
761             // 移进: dot < rhs.size() 且下一个符号是终结符
762             if (it.dot < (int)it.rhs.size() && isTerminal(it.rhs[it.dot])) {
763                 std::string a = it.rhs[it.dot];
764                 // 从 transMap 里查下一个状态 j
765                 auto key = std::make_pair(i, a);
766                 auto pos = transMap.find(key);
767                 if (pos != transMap.end()) {
768                     int j = pos->second;
769                     ACTION[i][a] = { Action::SHIFT, j };
770                 }
771             }
772             // 接受: 增广产生式 S' -> S ., gid==0 且 dot 到末尾
773             else if (it.dot == (int)it.rhs.size() && it.gid == 0) {
774                 ACTION[i]["$"] = { Action::ACCEPT, 0 };
775             }
776             // 规约: dot 在末尾 (但不是增广产生式)
777             else if (it.dot == (int)it.rhs.size()) {
778                 for (auto& a : it.lookahead) {
779                     ACTION[i][a] = { Action::REDUCE, it.gid };
780                 }
781             }
782         }
783
784         // 2.2) GOTO 部分: 对于所有非终结符转移
785         for (auto& nt : nonTerminals) {
786             auto key = std::make_pair(i, nt);
787             auto pos = transMap.find(key);
788             if (pos != transMap.end()) {
789                 GOTO_[i][nt] = pos->second;
790             }
791         }
792     }
793 }

```

图 3-4-3 LR(1)项目集转换图构造

完成上述的项目集转换图构造后，将每个项目集作为一个状态，借助 Qt 的 QGraphicsView 控件即可完成 LR(0)、LR(1) 的 DFA 图的绘制，并将图展示给用户查看

5) SLR(1)文法检测:

检测用户提供的文法是否为 SLR(1)文法，需要进行“移进-规约”和“规约-规约”这两种冲突的检测，这分别通过 SLR(1)_check1()和 SLR(1)_check2()函数实现:

```

795 // 检查移进-规约冲突
796 bool SLR1_check1()
797 {
798     for (int i = 0; i < (int)states.size(); ++i)
799     {
800         const State& st = states[i];
801         // 本状态上可移进的“终结符”
802         set<string> shift_terms;
803         for (auto& it : st)
804         {
805             if (it.dot < (int)it.rhs.size())
806             {
807                 const string& X = it.rhs[it.dot];
808                 // 只把真正的终结符算作 shift, 这里可以直接用 isTerminal
809                 if (isTerminal(X))
810                     shift_terms.insert(X);
811             }
812         }
813
814         // 对所有规约项目, 若它的 FOLLOW 集合与 shift_terms 有交集, 就冲突
815         for (auto& it : st)
816         {
817             if (it.dot == (int)it.rhs.size()) // 规约项目
818             {
819                 const string& B = grammar_deque[it.gid].left;
820                 for (auto& a : followSet[B])
821                 {
822                     if (shift_terms.count(a))
823                         return true;
824                 }
825             }
826         }
827     }
828     return false;
829 }

```

图 3-5-1 移进-规约冲突检测的代码实现

对于每个 LR(0)状态, 若某终结符同时在可移进符号集合某规约项目的 FOLLOW 集合中, 则存在移进-规约冲突

```

831 // 检查规约-规约冲突
832 bool SLR1_check2()
833 {
834     for (int i = 0; i < (int)states.size(); ++i)
835     {
836         const State& st = states[i];
837         // 收集本状态所有规约项目对应的非终结符
838         vector<string> reduces;
839         for (const auto& it : st)
840         {
841             if (it.dot == (int)it.rhs.size())
842                 reduces.push_back(grammar_deque[it.gid].left);
843         }
844
845         // 两两比较 FOLLOW 集合是否有交集
846         for (size_t p = 0; p + 1 < reduces.size(); ++p)
847         {
848             for (size_t q = p + 1; q < reduces.size(); ++q)
849             {
850                 const auto& F1 = followSet[reduces[p]];
851                 const auto& F2 = followSet[reduces[q]];
852                 for (const auto& a : F1)
853                 {
854                     if (F2.count(a))
855                         return true;
856                 }
857             }
858         }
859     }
860     return false;
861 }

```

图 3-5-2 规约-规约冲突检测的代码实现

对于每个 LR(0)状态, 若两条规约项目的 FOLLOW 集合有

交集，则存在规约-规约冲突

通过这两个函数的检测，即可判断用户输入的文法是否为 SLR(1)文法，并且在检测出并非 SLR(1)文法时，能够指出其原因，以下为 SLR(1)文法判断函数的代码：

```
863 // SLR(1)分析函数
864 int SLR1Analyse()
865 {
866     bool flag1 = SLR1_check1();
867     bool flag2 = SLR1_check2();
868     //移进规约、规约规约冲突
869     if(flag1 && flag2) return 3;
870     // 规约规约冲突
871     else if(flag2) return 2;
872     // 移进规约冲突
873     else if(flag1) return 1;
874     // 无冲突，是SLR(1)文法
875     else return 0;
876 }
```

图 3-5-3 SLR(1)文法判断的代码实现

```
1292 int status = SLR1Analyse();
1293 switch (status)
1294 {
1295     case 0: res = "该文法是 SLR(1) 文法"; break;
1296     case 1: res = "该文法发生移进-规约冲突，不是SLR(1)文法"; break;
1297     case 2: res = "该文法发生规约-规约冲突，不是SLR(1)文法"; break;
1298     case 3: res = "该文法发生移进-规约冲突和规约-规约冲突，不是SLR(1)文法"; break;
1299     default: res="未知错误"; break;
1300 }
1301 QString qres = QString::fromStdString(res);
1302 ui->plainTextEdit_check->setPlainText(qres);
```

图 3-5-3 判断结果显示

6) LR(1)分析表构建:

根据 LR(1)DFA 构建分析表 ACTION 和 GOTO，以下这两部分的内容及数据存储结构：

ACTION 表：

- 项目 $[A \rightarrow \alpha . a \beta]$ 且 a 为终结符，设为移进；
- 项目 $[S' \rightarrow S .]$ 设为接受；
- 项目 $[A \rightarrow \alpha .]$ 设为规约，对应 lookahead 符号

```

137 // ACTION[state][terminal] = {类型, 编号}
138 // GOTO[state][nonterminal] = 目标状态
139 // ACTION表
140 struct Action {
141     enum {SHIFT, REDUCE, ACCEPT, ERROR} act;
142     int num;
143 };
144 vector<unordered_map<string, Action>> ACTION;

```

图 3-6-1 ACTION 表存储结构

GOTO 表：对所有非终结符转移填入目标状态

```

146 // GOTO表
147 vector<unordered_map<string, int>> GOTO_;

```

图 3-6-2 GOTO 表存储结构

通过 buildLR1Table()函数，实现 LR(1)分析表的构建，代码如下：

```

745 // 构建LR1分析表
746 void buildLR1Table() {
747     int N = states_LR1.size();
748     ACTION.assign(N, {});
749     GOTO_.assign(N, {});
750
751     // 1) 先把 trans1 中的所有 (from, symbol)->to 映射缓存起来
752     std::map<std::pair<int, std::string>, int> transMap;
753     for (auto& t : trans) {
754         transMap[{t.from, t.symbol}] = t.to;
755     }
756
757     // 2) 填 ACTION 和 GOTO 表
758     for (int i = 0; i < N; ++i) {
759         // 2.1) ACTION 部分
760         for (auto& it : states_LR1[i]) {
761             // 移进: dot < rhs.size() 且下一个符号是终结符
762             if (it.dot < (int)it.rhs.size() && isTerminal(it.rhs[it.dot])) {
763                 std::string a = it.rhs[it.dot];
764                 // 从 transMap 里查下一个状态 j
765                 auto key = std::make_pair(i, a);
766                 auto pos = transMap.find(key);
767                 if (pos != transMap.end()) {
768                     int j = pos->second;
769                     ACTION[i][a] = { Action::SHIFT, j };
770                 }
771             }
772             // 接受: 增广产生式 S' -> S .., gid==0 且 dot 到末尾
773             else if (it.dot == (int)it.rhs.size() && it.gid == 0) {
774                 ACTION[i]["$"] = { Action::ACCEPT, 0 };
775             }
776             // 规约: dot 在末尾 (但不是增广产生式)
777             else if (it.dot == (int)it.rhs.size()) {
778                 for (auto& a : it.lookahead) {
779                     ACTION[i][a] = { Action::REDUCE, it.gid };
780                 }
781             }
782         }
783
784         // 2.2) GOTO 部分: 对于所有非终结符转移
785         for (auto& nt : nonTerminals) {
786             auto key = std::make_pair(i, nt);
787             auto pos = transMap.find(key);
788             if (pos != transMap.end()) {
789                 GOTO_[i][nt] = pos->second;
790             }
791         }
792     }
793 }

```

图 3-6-3 LR(1)分析表构建的代码实现

LR(1)分析表构建完成后，通过 Qt 控件 TableView 将构建号的分析表以表格的形式展示给用户查看

7) LR(1)句子分析:

用户通过键盘输入要分析的句子，程序通过处理用户给的文法规则，构建 LR(1)项目集，然后对用户输入的句子进行 LR(1)分析，将分析处理的过程通过表格的形式展现出来，代码如下：

```
1368 // 选做部分
1369 void doLR1Parse(const QStringList& inputTokens, QStandardItemModel *parseModel)
1370 {
1371     // 构建分析表
1372     getLR1();
1373     buildLR1Table();
1374
1375     // 初始化：状态栈、符号栈、输入指针
1376     vector<int> stateStack;
1377     QStringList symbolStack;
1378     stateStack.push_back(0);
1379     symbolStack.push_back("$");
1380
1381     // 输入指针
1382     int ip = 0;
1383     QStringList tokens = inputTokens;
1384
1385     while (true) {
1386         int curState = stateStack.back();
1387         QString a = tokens[ip];
1388
1389         // 查 ACTION 表
1390         auto itAct = ACTION[curState].find(a.toStdString());
1391         QString actionStr;
1392         if (itAct == ACTION[curState].end()) {
1393             actionStr = "error";
1394         } else {
1395             const Action& act = itAct->second;
1396             switch (act.act) {
1397                 case Action::SHIFT:
1398                     actionStr = QString("s%i").arg(act.num);
1399                     // 记录当前行
1400                     parseModel->appendRow(QList<QStandardItem*>{
1401                         new QStandardItem(symbolStack.join(" ") + " | " + QString::number(stateStack.back())),
1402                         new QStandardItem(tokens.mid(ip).join(" ")),
1403                         new QStandardItem(actionStr)
1404                     });
1405                     // 执行 shift
1406                     symbolStack.push_back(a);
1407                     stateStack.push_back(act.num);
1408                     ip++;
1409                     break;
1410
1411                 case Action::REDUCE: {
1412                     int rid = act.num;
1413                     auto& prod = grammar_deque[rid];
1414                     // 右部符号数
1415                     int k = splitSymbols(prod.right).size();
1416                     // 记录当前行
1417                     parseModel->appendRow(QList<QStandardItem*>{
1418                         new QStandardItem(symbolStack.join(" ") + " | " + QString::number(stateStack.back())),
1419                         new QStandardItem(tokens.mid(ip).join(" ")),
1420                         new QStandardItem(QString("r%i(%2-%3)")
1421                             .arg(rid)
1422                             .arg(QString::fromStdString(prod.left))
1423                             .arg(QString::fromStdString(prod.right)))
1424                     });
1425                     // 执行 reduce: 弹出 k 个符号和 k 个状态
1426                     for (int i = 0; i < k; ++i) {
1427                         symbolStack.pop_back();
1428                         stateStack.pop_back();
1429                     }
1430                     // 压入左部非终结符
1431                     QString A = QString::fromStdString(prod.left);
1432                     symbolStack.push_back(A);
1433                     // GOTO
1434                     int gotoState = GOTO_[stateStack.back()][prod.left];
1435                     stateStack.push_back(gotoState);
1436                     break;
1437                 }
1438
1439                 case Action::ACCEPT:
1440                     // 记录最后一行
1441                     parseModel->appendRow(QList<QStandardItem*>{
1442                         new QStandardItem(symbolStack.join(" ") + " | " + QString::number(stateStack.back())),
1443                         new QStandardItem("$"),
1444                         new QStandardItem("acc")
1445                     });
1446                     QMessageBox::information(nullptr, "结果", "句子被成功接受!");
1447                     return;
1448
1449                 default:
1450                     QMessageBox::critical(nullptr, "错误", "出现无法识别的动作!");
1451                     return;
1452             }
1453         }
1454     }
1455 }
```

图 3-7-1、图 3-7-2 选做内容的代码实现

4. 系统实现

4.1 用户界面设计

4.1.1 布局：

主窗口包含使用说明区、文本输入区、FIRST&FOLLOW 集合表格化展示区、DFA 图展示区、SLR(1)文法判断区、LR(1)分析展示区共 6 个模块，使用说明区告诉用户文法和句子的输入格式以及显示内容格式；文本输入区供用户输入文法规则和语句，文法规则可通文件对话框选择载入，还可以由用户选择以文件的形式保存到本地；FIRST&FOLLOW 集合表格化展示区可以让用户查看当前文法的所有非终结符号（串）的 first 集合和 follow 集合；DFA 图展示区可供用户查看 LR(0)、LR(1)DFA 图；SLR(1)文法判断区可以帮助用户判断其输入的文法规则是否为 SLR(1)文法，若不是则能指出其原因；LR(1)分析展示区可以让用户查看当前文法的 LR(1)分析表，并还能查看用户输入的句子的 LR(1)逐步分析结果。按钮【读取文法规则】实现通过文件对话框选取文法规则文件；按钮【保存文法规则】实现将当前输入在输入框中的文法规则保存下来；按钮【查看使用说明】可以查看本软件的使用说明；按钮【查看 first 集合和 follow 集合】借助 Qt 控件 TableView 实现当前文法的所有非终结符号（串）的 first 集合和 follow 集合的表格化展示；按钮【查看 LR(1)分析表】借助 Qt 控件 TableView 实现 LR(1)分析表的展示；按钮【查看 LR(0)DFA】借助 Qt 控件 GraphicsView 实现 LR(0)DFA 图的展示；按钮【查看 LR(1)DFA】借助 Qt 控件 GraphicsView 实现 LR(1)DFA 图的展示；按钮【LR(1)句子分析】借助 Qt 中的 TableView 控件输出语句经 LR(1)分析的过程表格

主窗口界面如下图所示：

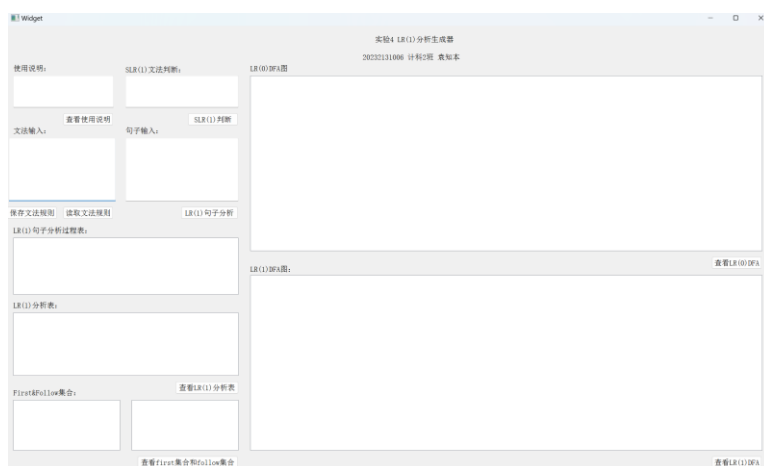


图 4-1-1 主窗口控件布局

4.1.2 交互逻辑：

点击按钮【读取文法规则】，即可打开文件对话框，在文件对话框中选择你要导入的文法规则文件，单击【确定】即可导入

示例如下：

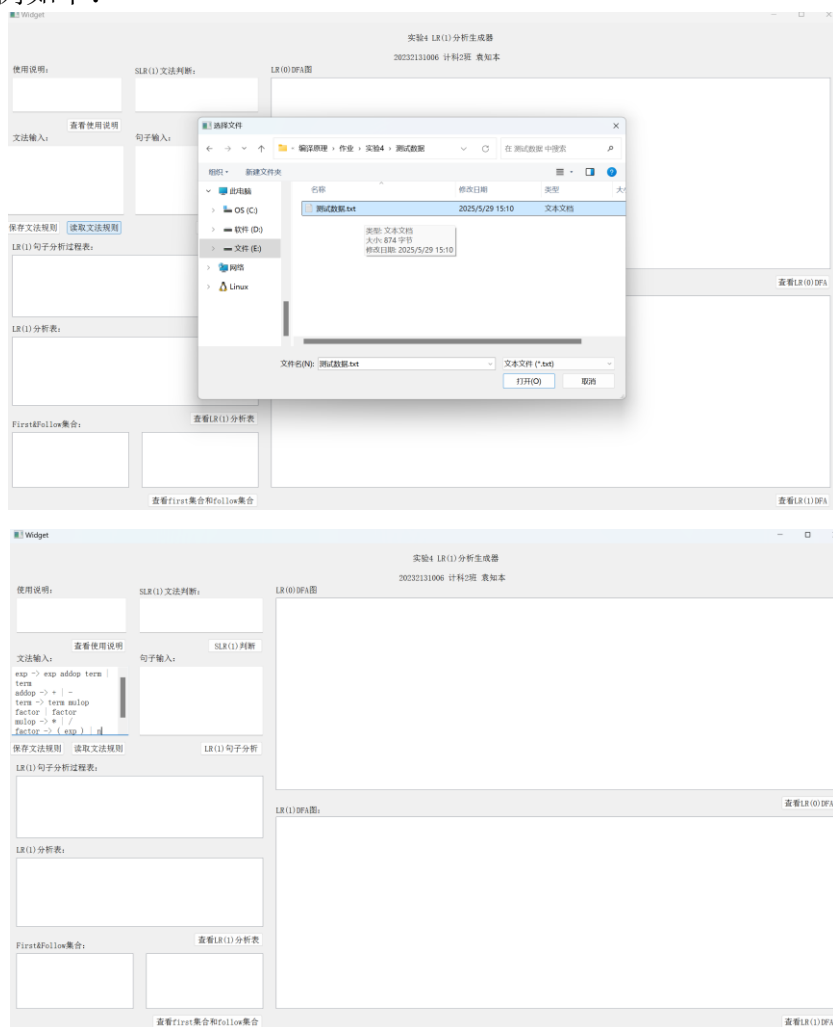


图 4-1-2、图 4-1-3 文法规则导入功能展示

如上图所示，除了直接输入的方式导入文法规则，也可以通过点击【读取文法规则】实现通过打开文件对话框，选择本地的文法规则导入

在导入或手动输入了文法规则后，点击按钮【保存文法规则】，即可打开文件对话框，自定义保存的文件名、保存的位置路径，点击【保存】即可将当前输入到文法规则输入框中的文法规则保存到你刚才指定的文件和位置中，其展示分别如下：

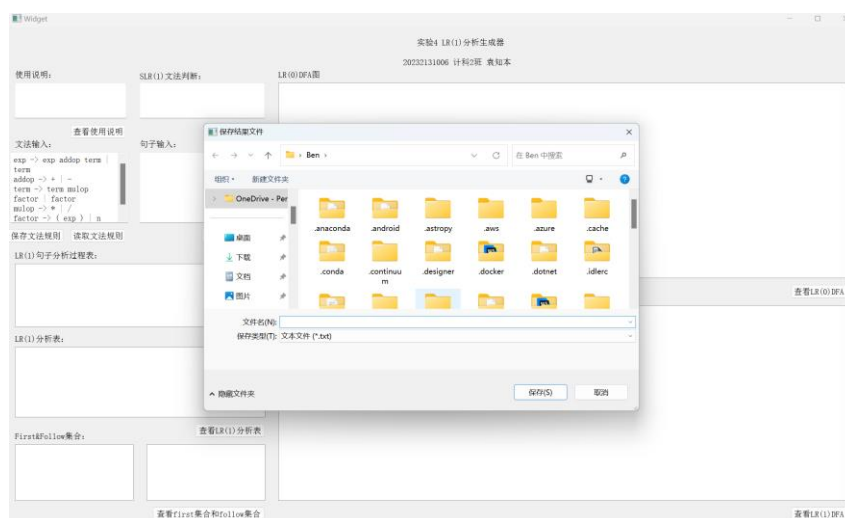


图 4-1-4 保存文法规则功能演示（打开文件对话框）

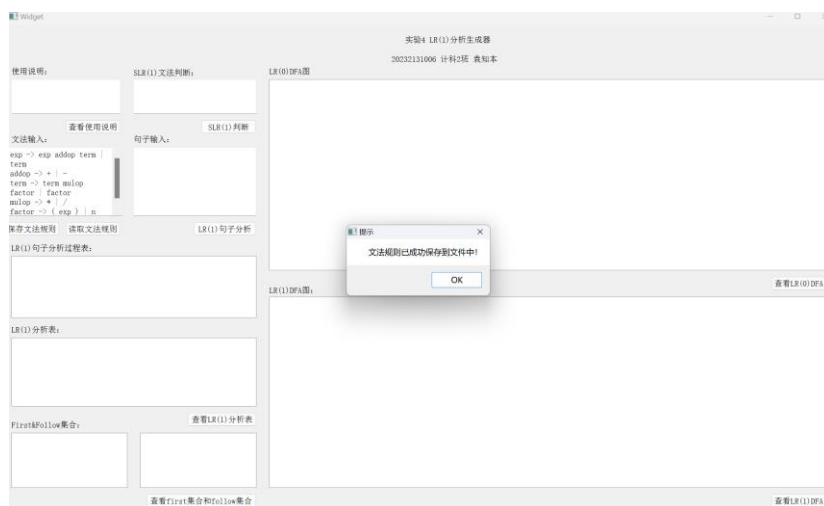


图 4-1-5 保存文法规则功能演示（保存成功的提示）

在导入或手动输入了文法规则后，点击按钮【查看 first 集合和 follow 集合】，系统将会将这段文法规则暂存到字符串变量 `grammar_str` 中，然后开始根据“按行拆分->解析每条文法规则（区分 LHS 和 RHS，处理分支）->增广处理->编号”的过程进行文法规则处理，处理完成后将会开始构建 first 集合和 follow 集合，构造完成后将 first 集合和 follow 集合借助 Qt 中的 TableView 控件输出显示，运行效果如下：

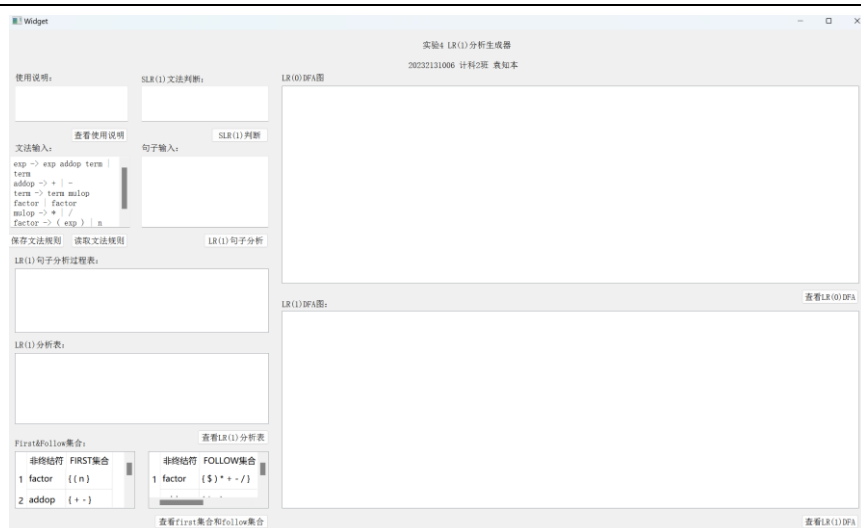


图 4-1-6 first 集合和 follow 集合构建与展示功能演示

接下来，点击按钮【SLR(1)判断】，可以对当前输入的文法进行 SLR(1)文法判断——即判断当前文法是否为 SLR(1)文法：



图 4-1-7 是 SLR(1)文法

Widget

使用说明:

SLR(1) 文法判断:

该文法发生移进-规约冲突，不是SLR(1) 文法

查看使用说明

SLR(1) 判断

文法输入:

S $\rightarrow$ A a | A b
A $\rightarrow$ ϵ | a

句子输入:

保存文法规则

读取文法规则

LR(1) 句子分析

图 4-1-8 不是 SLR(1)文法（发生“移进-规约”冲突的情况）

Widget

使用说明:

SLR(1) 文法判断:

该文法发生规约-规约冲突，不是SLR(1) 文法

查看使用说明

SLR(1) 判断

文法输入:

S $\rightarrow$ A
A $\rightarrow$ a
A $\rightarrow$ a

句子输入:

保存文法规则

读取文法规则

LR(1) 句子分析

图 4-1-9 不是 SLR(1)文法（发生“规约-规约”冲突的情况）



图 4-1-10 不是 SLR(1)文法（同时发生“移进-规约”和“规约-规约”冲突的情况）

之后，我们按下按钮【查看 LR(0)DFA】可以查看到当前文法的 LR(0)DFA 图：

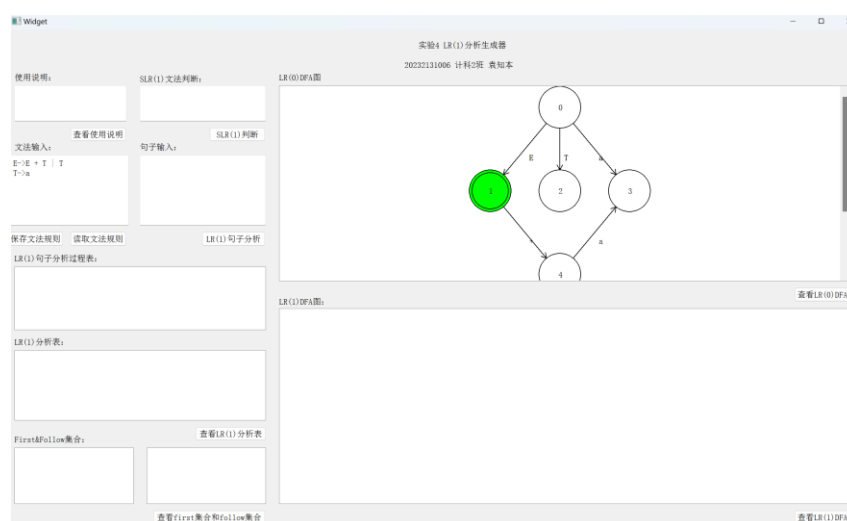


图 4-1-11 LR(0)DFA 图展示

绿色双圈的状态表示 acc 情况，由于显示框的大小，在不拖动滚动条下拉的情况下对当前文法的 LR(0)DFA 图可能显示不全，但查看完整的图可以得知其完全正确

现在按下按钮【查看 LR(1)DFA】可以查看到当前文法的 LR(1)DFA 图：

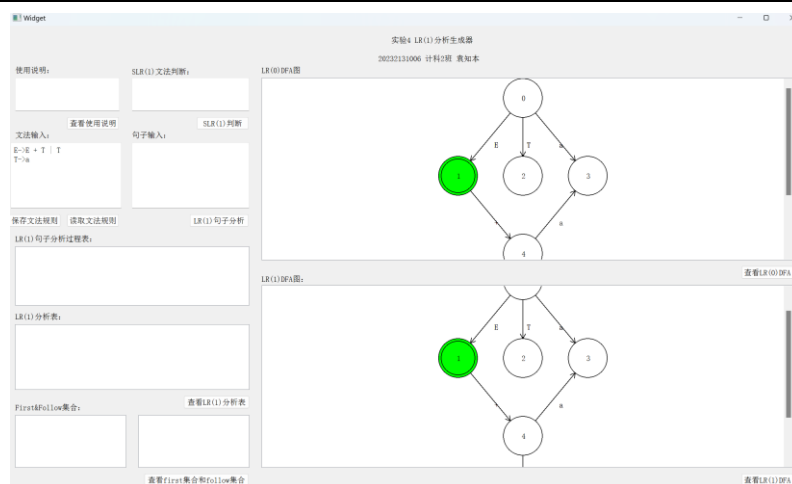


图 4-1-12 LR(1)DFA 图展示

绿色双圈的状态表示 **acc** 情况，由于显示框的大小，在不拖动滚动条下拉的情况下对当前文法的 LR(1)DFA 图可能显示不全，但查看完整的图可以得知其完全正确

现在按下按钮【查看 LR(1)分析表】，即可查看到当前文法对应的 LR(1)分析表：

State	\$	+	a	E	T
1	0		s3	1	2
2	1	acc	s4		
3	2		r2(E→T)		r2(E→T)

图 4-1-13 LR(1)分析表展示

接下来是选做内容：

在输入了语法规则之后，在句子输入框中输入句子，点击按钮【LR(1)句子分析】，即可查看逐步分析输入的句子表格，并且能够指出该句子是否符合当前的语法规则：

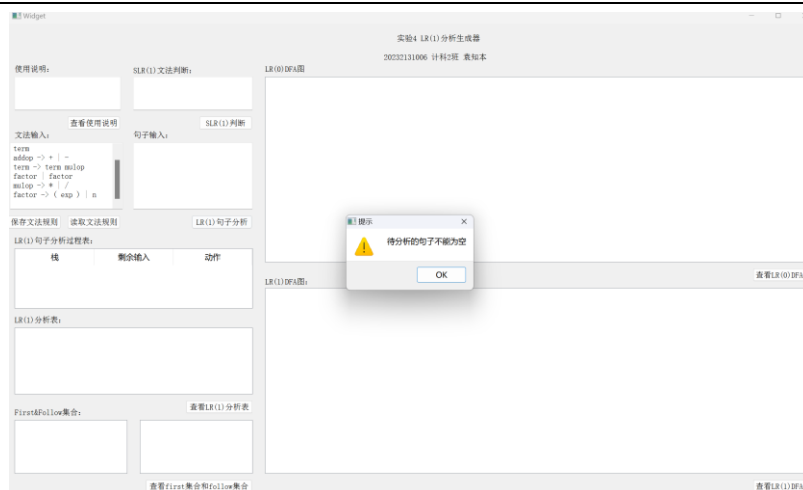


图 4-1-14 不输入任何句子内容的情况

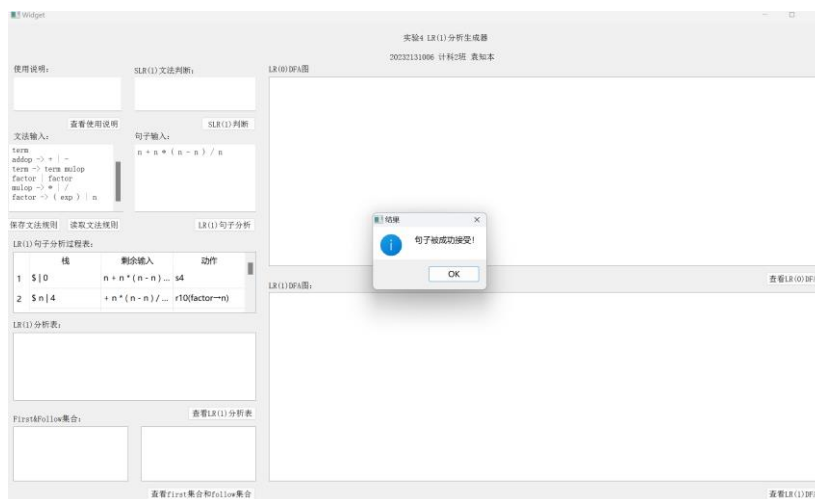


图 4-1-15 输入句子符合语法规则的情况

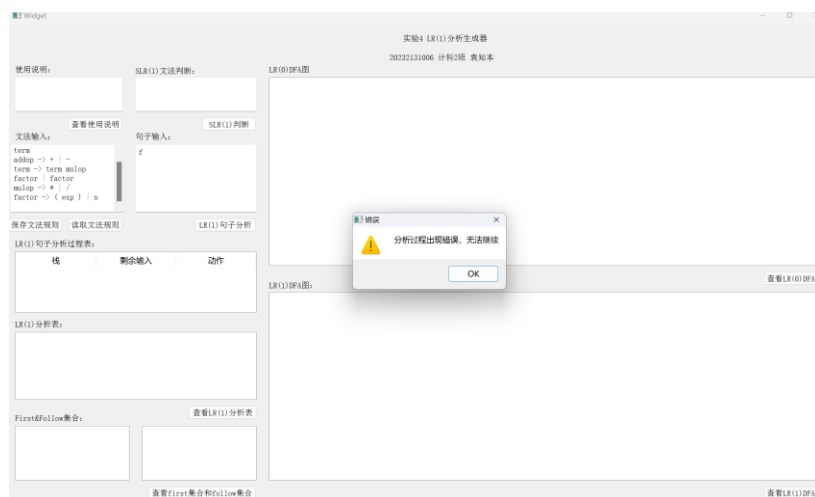


图 4-1-16 输入句子不符合语法规则的情况

4.1.3 用户交互界面的实现:

控件的实现:

如图 4-1-9 所示, 用户交互界面的有按钮、文本编辑框和树型控件等若干控件, 均可通过 Qt 的工具栏拖拽进行添加, 然后给文本编辑框和树型控件等控件设置对应的变量, 对应的按钮设置响应函数, 其代码如下图所示:

```
14 public:
15     Widget(QWidget *parent = nullptr);
16     ~Widget();
17
18 private slots:
19     // 载入文法规则文件的按钮控件
20     void on_pushButton_load_clicked();
21     // 保存文法规则文件的按钮控件
22     void on_pushButton_save_clicked();
23     // 使用说明的按钮控件
24     void on_pushButton_help_clicked();
25     // 显示first集合和follow集合的按钮控件
26     void on_pushButton_firstandfollow_clicked();
27     // 判断文法是否为SLR(1)文法的按钮控件
28     void on_pushButton_check_clicked();
29     // 生成LR(0)DFA图的按钮控件
30     void on_pushButton_LR0DFA_clicked();
31     // 生成LR(1)DFA图的按钮控件
32     void on_pushButton_LR1DFA_clicked();
33     // 生成LR(1)分析表的按钮控件
34     void on_pushButton_LR1table_clicked();
35     // 对语句进行LR(1)分析并生成分析表的按钮控件
36     void on_pushButton_sentence_clicked();
37
38
39 private:
40     Ui::Widget *ui;
```

图 4-1-9 响应函数一览

```

26     public:
27         QLabel *label;
28         QLabel *label_2;
29         QLineEdit *plainTextEdit;
30         QLabel *label_3;
31         QPushButton *pushButton_save;
32         QPushButton *pushButton_load;
33         QGraphicsView *graphicsView_LR1;
34         QLabel *label_4;
35         QGraphicsView *graphicsView_LR0;
36         QLabel *label_5;
37         QTableView *tableView_LR1;
38         QLabel *label_6;
39         QLineEdit *plainTextEdit_check;
40         QLabel *label_7;
41         QPushButton *pushButton_check;
42         QPushButton *pushButton_LR1table;
43         QPushButton *pushButton_LR0DFA;
44         QPushButton *pushButton_LR1DFA;
45         QPushButton *pushButton_help;
46         QLineEdit *plainTextEdit_help;
47         QLabel *label_8;
48         QLabel *label_10;
49         QPushButton *pushButton_firstandfollow;
50         QTableView *tableView_firstSet;
51         QTableView *tableView_followSet;
52         QTableView *tableView_LR1Tokens;
53         QLabel *label_9;
54         QLineEdit *plainTextEdit_sentence;
55         QLabel *label_11;
56         QPushButton *pushButton_sentence;

```

图 4-1-10 控件变量一览

5. 系统测试

5-1 系统启动

由于已经生成了 Release 版本，直接双击可执行程序即可打开：

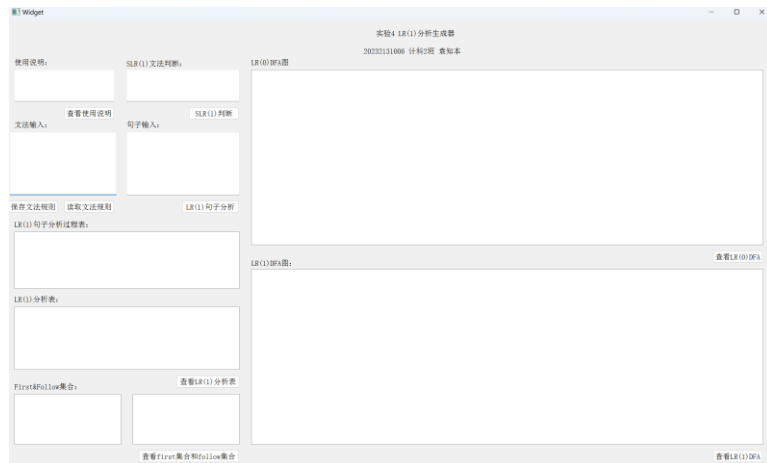


图 5-1-1 系统运行成功截图

5-2 数据测试

一共有 5 组测试，其测试样例与测试结果如下图所示：

测试 1：

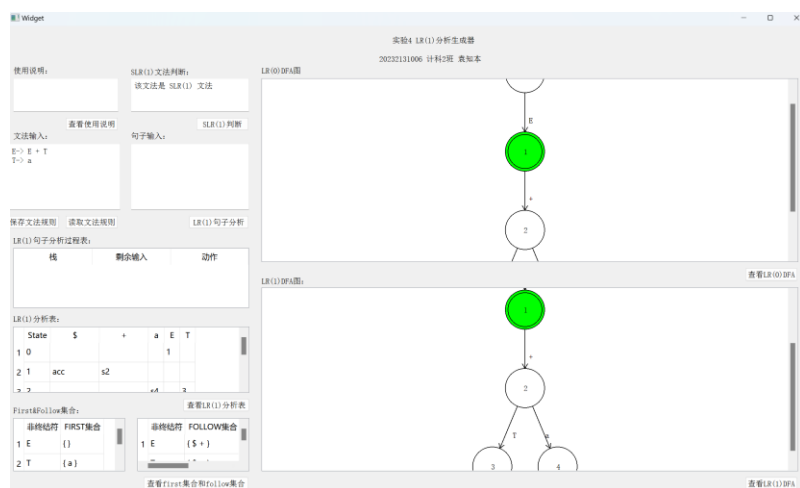


图 5-2-1 测试 1 成功

测试 2：

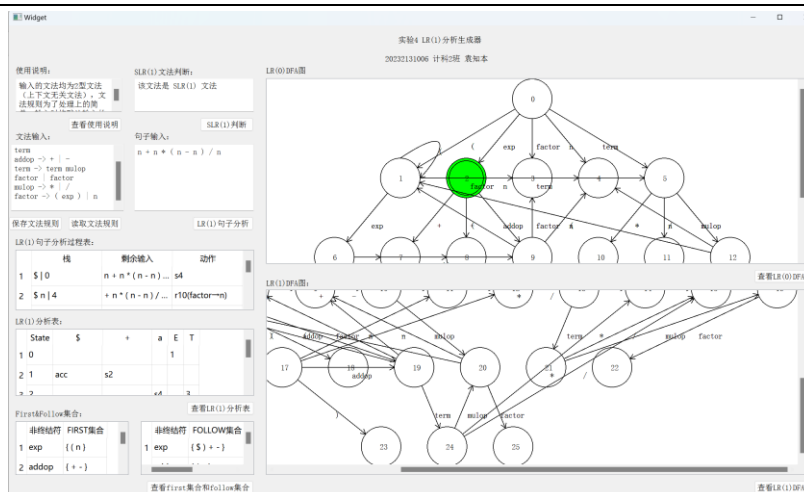


图 5-2-2 测试 2 成功

测试 3:

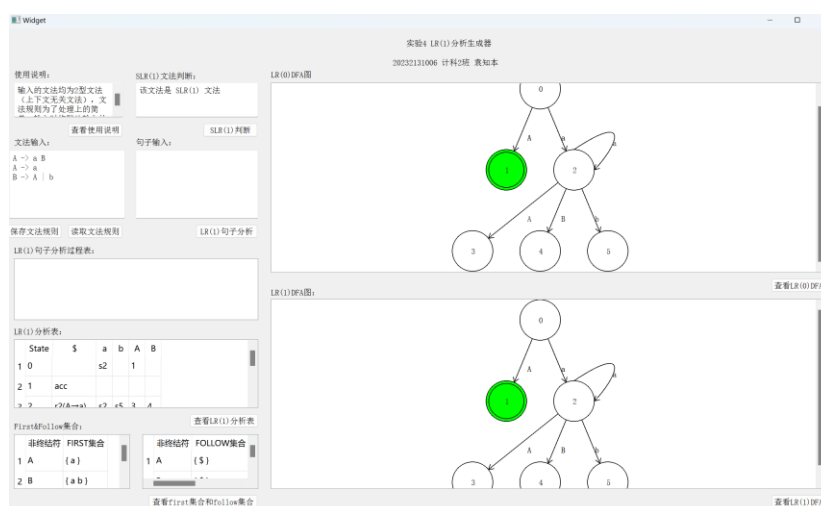


图 5-2-3 测试 3 成功

测试 4:

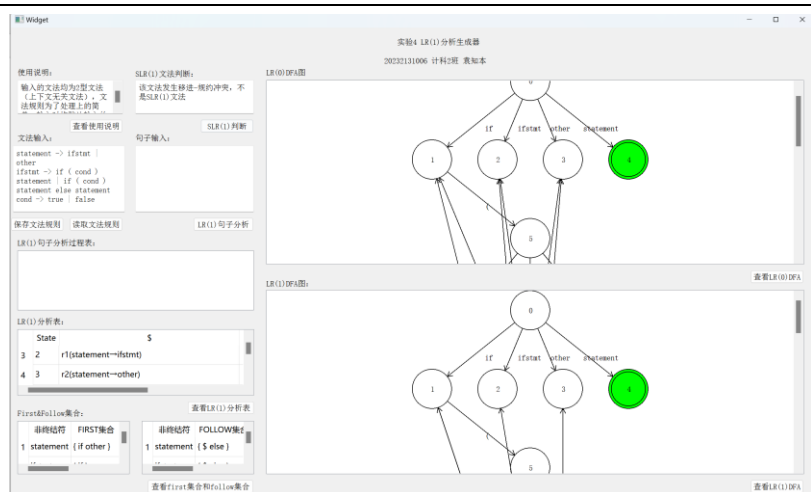


图 5-2-4 测试 4 成功

测试 5:

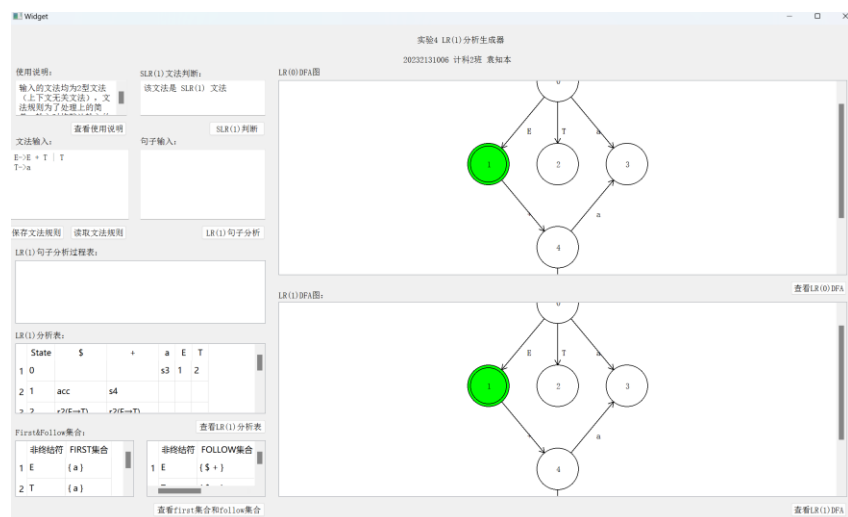


图 5-2-5 测试 5 成功

综上，根据测试数据中的内容进行各项测试，其结果均显示正确，结果放在测试结果文件夹中

四、实验总结（心得体会）

本次实验，我继续使用 Qt 进行图形化界面的实现，并使用了更多新的 Qt 的控件，对 Qt 可视化界面开发有了更深的了解，提高了我的编程能力，为我日后工作提供了宝贵的实战经验

在本次实验中，通过反复调试 FIRST 和 FOLLOW 的集合计算、项

目集闭包与转移动作，我对相关的理论细节更加清晰，在解决本次实验的过程中，我也提高了自身阅读代码、分析代码的能力

综上所述，本次实验锻炼了我阅读代码能力、理解代码功能能力、在新环境和条件下复现程序功能的能力以及在时间紧张情况下完成任务的抗压能力，加深了我对编程语言设计、软件开发的理解，获益良多

五、参考文献：

[【编译原理】一篇搞定 LR 分析法（LR\(1\)、LR\(0\)、SLR、LALR） - CSDN 博客](#)

[3 编译原理如何求 first 集和 follow 集（更正版） 哔哩哔哩 bilibili](#)

[7 编译原理构造 LR\(0\)分析表\(包含构建项目规范族\) 哔哩哔哩 bilibili](#)

[9 编译原理构造 LR\(1\)分析表（带向前搜索符的项目集规范族） 哔哩哔哩 bilibili](#)

[11 编译原理如何区别 LR\(0\),SLR\(1\),LR\(1\),LALR\(1\)文法 哔哩哔哩 bilibili](#)